

LAB211 Assignment

Type:	Long Assignment
Code:	J1.L.P0036
LOC:	300
Slot(s):	N/A

Title

Football Club & Player Management

Background

A football tournament called the European Elite League (EEL) needs to manage information about participating clubs and players. Therefore, they hired FTC Software Company, where you work as a professional Java developer, to implement this project as a console-based application.

The system stores data in two text files, automatically loads them on startup, and provides a thirteenth function that allows users to reload the data on demand:

- **clubs.txt** – containing football club information
- **players.txt** – containing player information

Students are required to analyze and design the program using an Object-Oriented Programming (OOP) approach. The development process must incorporate OOP principles, such as abstraction, polymorphism, encapsulation, and inheritance.

File clubs.txt	Description
CL-0001, FC Barcelona, Nike, 820 CL-0002, Real Madrid Galácticos, Adidas, 910 CL-0003, Manchester City Elite, Puma, 880 CL-0004, Bayern Munich Prime, Adidas, 860 CL-0005, Liverpool FC, Nike, 740 CL-0006, Paris Saint-Germain, Nike, 890 CL-0007, Juventus FC, Adidas, 700 CL-0008, AS Roma FC, Adidas, 720 CL-0009, AC Milan, Puma, 610 CL-0010, AC Milan Sportline, Puma, 630 CL-0011, Chelsea FC, Nike, 760 CL-0012, Chelsea FC XLine, Nike, 780	Information on a line: <Club ID, club name, sponsor brand, budget> budget is a numeric value in million EUR (e.g., 820 means 820 million EUR).

File players.txt	Description
P0001, CL-0001, Lionel Messi, Forward, 10 P0002, CL-0001, Sergio Busquets, Midfielder, 5 P0003, CL-0001, Jordi Alba, Defender, 18 P0004, CL-0002, Luka Modrić, Midfielder, 10	Information on a line: <Player ID, club ID, player name, position, shirt number>

P0005, CL-0002, Toni Kroos, Midfielder, 8 P0006, CL-0002, Rodrygo Silva, Forward, 21 P0007, CL-0003, Erling Haaland, Forward, 9 P0008, CL-0003, Kevin De Bruyne, Midfielder, 17 P0009, CL-0004, Thomas Müller, Forward, 25 P0010, CL-0004, Joshua Kimmich, Midfielder, 6 P0011, CL-0005, Mohamed Salah, Forward, 11 P0012, CL-0005, Virgil van Dijk, Defender, 4 P0013, CL-0012, Raheem Sterling, Winger, 7 P0014, CL-0012, Enzo Fernández, Midfielder, 8 P0015, CL-0012, Reece James, Defender, 24	
---	--

Constraints

1- Constraints on Clubs

- a. Club ID must be unique
- b. Club ID format: CL-xxxx (e.g., CL-0001)
- c. Club name cannot be empty
- d. Sponsor brand cannot be empty
- e. Budget must be a positive real number

2- Constraints on Players

- a. Player ID must be unique
- b. Player ID format: Pxxxx (e.g., P0001)
- c. Club ID must exist in the club list
- d. Player name cannot be empty
- e. Position must be one of the following values: Goalkeeper, Defender, Midfielder, Forward, Winger (case-insensitive).
- f. Shirt number must be an integer between 1 and 99 and unique within the same club

Program Specifications

The program's menu system includes 14 functions that allow users to control the application, as described below.

1. List of all clubs
2. Add a new club
3. Search for a club by ID
4. Update a club by ID
5. List of all clubs with budget $\leq$ input value
6. List all players in ascending order of club names, if same club, sort by shirt number ascending.
7. Search players by partial player name

8. Add a new player
9. Remove a player with ID
10. Update a player with an ID
11. List all players by a specific position
12. Save data to files
13. Load data from files
14. Quit program

Features and Assessment:

No	LOCs	Function
1/	14	List all clubs: Display data in a table with column headers (Club ID, Club Name, Sponsor Brand, Budget).
2/	35	Add a new club: Accept club ID, club name, sponsor brand, and budget; add to the list if constraints are satisfied. If the club ID already exists, show: <i>“This club ID already exists!”</i> . Display success/failure message.
3/	14	Search for a club by ID: If the club does not exist, show: <i>“This club does not exist!”</i> . Otherwise, display club information.
4/	35	Update a club by ID: Accept club ID. If the club does not exist, show: <i>“This club does not exist!”</i> . Otherwise, allow updating club name, sponsor brand, and budget. Empty input means skipping the update for that field.
5/	35	List all clubs with budgets less than or equal to an input value: Accept a budget input; display all clubs with budget $\leq$ input value.
6/	14	List all players in ascending order of club names: Display all players sorted by club names in ascending order; if players belong to the same club, sort by shirt number ascending.
7/	56	Search players by partial player name match: Accept partial player name (e.g., <i>Mess</i>); display all matching players (<i>partial name match is case-insensitive</i>).
8/	56	Add a new player: Accept player ID, club ID (<i>display club list and let user input an existing club ID</i>), player name, position, and shirt number. Validate all constraints before adding, including shirt number uniqueness within the same club. If the player ID already exists, show: <i>“This player ID already exists!”</i> . If the club ID does not exist, show: <i>“This club does not exist!”</i> . If the shirt number already exists within the same club, show: <i>“This shirt number already exists in this club!”</i> .

9/	14	Remove a player with ID: Accept player ID. If the player does not exist, show: “ <i>This player does not exist!</i> ”. Otherwise, remove it from the list.
10/	28	Update a player by ID: Accept player ID. If the player does not exist, show: “ <i>This player does not exist!</i> ”. Otherwise, allow updating player name, position, and shirt number. Empty input means skipping the update for that field. When updating shirt number, validate that it is unique within the same club; if not, show: “ <i>This shirt number already exists in this club!</i> ”.
11/	14	List all players by a specific position: Accept a position and display all players playing in that position.
12/	14	Save data to files: Save both club and player lists into clubs.txt and players.txt. Show confirmation message after completion.
13/	14	Load data from files: Clear current data, load clubs then players. Validate strict constraints. If any invalid line exists, show “ <i>Load data failed!</i> ”; otherwise show “ <i>Load data successfully!</i> ”.
14/	7	Quit program: If changes were made to clubs or players, save updated lists before exiting the program.

Note for Students:

- **Follow OOP principles** (encapsulation, inheritance, polymorphism, abstraction).
- **Follow Computational Thinking** (Decomposition, Pattern Recognition, Abstraction, and Algorithm) for reporting and presentation.
- **Input validation must strictly follow constraints.**
- **Show appropriate messages for invalid or missing data.**
- **Evaluation: Understanding, Correctness, data validation, and coding style. - Submission**
 - **Source code**
 - **Diagram**
 - **Flow-chart**